Phân tích 3 Cổng của LSTM (Phần 1)

Dựa vào sơ đồ **Hình 15-12**:

- **1. Cổng Quên (Forget Gate - $f_{(t)}$):**
  - Nhận vào $h_{(t-1)}$ và $x_{(t)}$, tính toán bằng Sigmoid.
  - Quyết định phần nào của Trạng thái dài hạn $c_{(t-1)}$ sẽ bị "bỏ đi". (Ví dụ: Thấy chủ ngữ mới thì quên chủ ngữ cũ).
- **2. Cổng Cập nhật & Ghi (Input Gate - $i_{(t)}, g_{(t)}$):**
  - $g_{(t)}$ tạo ra một vector giá trị ứng viên mới (dùng $\tanh$).
  - $i_{(t)}$ quyết định (bằng Sigmoid) mức độ quan trọng của các ứng viên này để cộng vào $c_{(t)}$ làm thông tin mới.

Phân tích 3 Cổng của LSTM (Phần 2)

- **Cập nhật Trạng thái Tế bào ($c_{(t)}$):**

$$c_{(t)} = f_{(t)} \otimes c_{(t-1)} + i_{(t)} \otimes g_{(t)}$$

(Quên đi quá khứ không cần thiết + Nhớ thêm thông tin mới).

- **3. Cổng Xuất (Output Gate - $o_{(t)}$):**
  - Quyết định phần nào của Trạng thái Tế bào $c_{(t)}$ sẽ được đọc ra và đưa vào Trạng thái ẩn $h_{(t)}$.
  - $o_{(t)} = \sigma(W_o[x_{(t)}, h_{(t-1)}])$
  - $h_{(t)} = o_{(t)} \otimes \tanh(c_{(t)})$

Sử dụng LSTM trong Keras

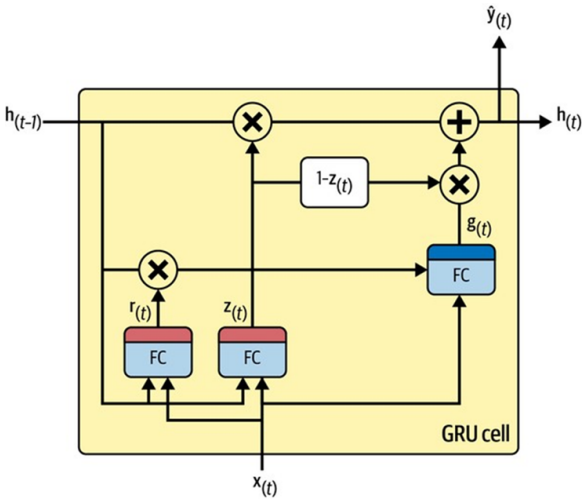
- Cực kỳ đơn giản, chỉ cần thay `SimpleRNN` bằng `LSTM`. Mọi công thức cổng phức tạp bên trên đã được Keras tối ưu bằng thư viện CUDNN dưới nền.

```
model = keras.Sequential([
    keras.layers.LSTM(20, return_sequences=True, input_shape=[None, 1]),
    keras.layers.LSTM(20),
    keras.layers.Dense(1)
])
```

- LSTM giải quyết tốt chuỗi dài khoảng hàng trăm bước thời gian, vượt xa SimpleRNN (chỉ hiệu quả với < 20 bước).

Tế bào GRU (Gated Recurrent Unit)

- GRU (2014) là một nỗ lực đơn giản hóa cấu trúc của LSTM.
- Hai điểm khác biệt chính:
  1. GRU không có đường ray $c_{(t)}$ riêng biệt. Nó gộp chung Trạng thái ẩn $h_{(t)}$ và Trạng thái tế bào làm một.
  2. Gộp Cổng Quên và Cổng Cập nhật thành một **Cổng Cập nhật (Update Gate - $z_{(t)}$)**. Nếu $z = 1$ thì lưu mới, nếu $z = 0$ thì giữ cũ.



Hình 15-13: Cấu tạo Tế bào GRU

So sánh LSTM và GRU

Đặc điểm	**LSTM**	**GRU**
Năm ra đời	1997	2014
Trạng thái duy trì	h_t và c_t	Chỉ h_t
Số lượng Cổng	3 (Quên, Nhập, Xuất)	2 (Cập nhật, Reset)
Tốc độ huấn luyện	Chậm	Nhanh hơn
Số lượng tham số	Lớn (Nặng RAM)	Thấp hơn ($\sim 25\%$ ít hơn)
Độ chính xác	Tốt với chuỗi rất phức tạp	Ngang ngửa LSTM ở đa số Data

- **Lời khuyên thực tiễn:** Bắt đầu thử nghiệm với GRU vì nó nhanh. Nếu thấy hiệu năng có vẻ bị thắt nút, chuyển sang LSTM xem có cải thiện không.

Tại sao lại dùng CNN cho Dữ liệu Chuỗi?

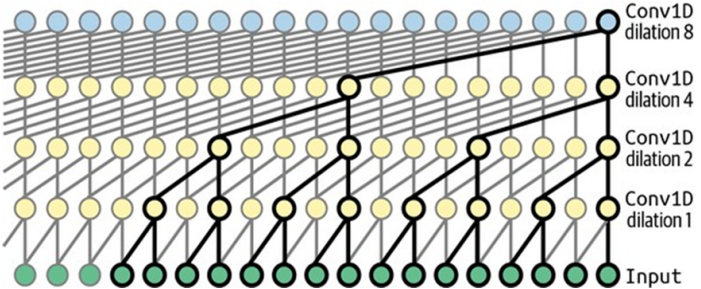
- RNN (ngay cả LSTM/GRU) có một điểm yếu chết người: **Tính toán tuần tự**.
- Bạn không thể tính bước thời gian $t = 100$ nếu chưa tính xong bước $t = 99$. Điều này khiến RNN **không thể chạy song song** trên phần cứng GPU một cách hiệu quả.
- Mạng Tích chập (CNN) với Kernel trượt thì độc lập với nhau, **hoàn toàn song song hóa** được.
- Ta có thể dùng **CNN 1D** (chỉ quét theo chiều dài thời gian) để trích xuất đặc trưng chuỗi.

Hạn chế của CNN 1D truyền thống

- Bộ lọc (Kernel) CNN 1D có kích thước nhỏ (VD: size 3).
- Nghĩa là mỗi nơ-ron lớp CNN 1 chỉ nhìn thấy 3 bước thời gian.
- Để nhìn được 100 bước thời gian (Trường thụ cảm - Receptive field lớn), ta phải xếp chồng hàng chục lớp CNN 1D, kết hợp với các lớp Pooling.
- Việc này rất nặng nề và làm mất đi thông tin chuỗi chi tiết. Cần một cơ chế quét khác!

Giải pháp đột phá: Kiến trúc WaveNet (2016)

- WaveNet do DeepMind phát triển, chuyên tạo giọng nói Audio. Âm thanh có tới 16,000 mẫu/giây $\Rightarrow$ Chuỗi cực kỳ dài!
- Thay vì CNN thường, nó sử dụng **Tích chập giãn nở (Dilated Convolutions)**.
- **Hình 15-14** minh họa: Tỷ lệ giãn (Dilation rate) tăng dần theo lũy thừa 2^n (1, 2, 4, 8...).



Hình 15-14: Kiến trúc WaveNet (Dilated Conv)

Cách hoạt động của Dilated Convolution

- Lớp 1 (rate=1): Kernel trượt từng ô liền kề (Giống hệt Conv1D thường).
- Lớp 2 (rate=2): Kernel bỏ qua 1 ô xen kẽ, giúp 1 nơ-ron bao phủ được khoảng cách 4 ô.
- Lớp 3 (rate=4): Kernel bỏ qua 3 ô xen kẽ, một nơ-ron nhìn được 8 ô.
- Nhờ tăng rate theo lũy thừa, mạng chỉ cần 10 lớp là đã có Trường thụ cảm bao phủ được **1024 bước thời gian** mà số lượng tham số vẫn cực kỳ nhỏ.

Code: Xây dựng WaveNet với Keras

Ta dùng vòng lặp để tạo các lớp `Conv1D` giãn nở:

```
model = keras.Sequential()
model.add(keras.layers.InputLayer(input_shape=[None, 1]))

# Cac lop tich chap gian no (1, 2, 4, 8) lap lai
for rate in (1, 2, 4, 8) * 2:
    model.add(keras.layers.Conv1D(
        filters=20, kernel_size=2, padding="causal",
        activation="relu", dilation_rate=rate))

model.add(keras.layers.Conv1D(filters=10, kernel_size=1))
model.add(keras.layers.Dense(1)) # Lop du doan
```

- Tham số `padding="causal"` cực kỳ quan trọng ở Time Series, nó chèn 0 vào bên trái để Kernel không bao giờ với tới thông tin của "tương lai".

Tổng Kết Chương 15 (Phần 1)

- **Sequence Data:** Đòi hỏi mô hình có bộ nhớ để theo dõi thứ tự.
- **RNN Cơ bản:** Chia sẻ trọng số qua các bước thời gian. Dùng Trạng thái ẩn $h_{(t)}$ để mang thông tin tới tương lai.
- **Ứng dụng RNN đa dạng:** Seq2Seq (Dự báo, Dịch), Seq2Vec (Phân tích cảm xúc), Encoder-Decoder (Chatbot).
- **BPTT (Backpropagation Through Time):** Mở cuộn mạng qua không gian thời gian. Dễ gây ra hiện tượng Gradient bốc hơi hoặc bùng nổ khi chuỗi quá dài.

Tổng Kết Chương 15 (Phần 2)

- **Tiền xử lý chuỗi (Differencing):** Rất hữu ích để loại bỏ tính thời vụ và xu hướng trước khi đưa vào mạng học sâu.
- **LSTM & GRU:** Thiết kế Tế bào thông minh với các "Cổng"(Gate) động đóng mở bằng Sigmoid, giúp bảo toàn Gradient đi xa, mang lại "Trí nhớ Dài Hạn".
- **CNN 1D & WaveNet:** Một hướng đi song song (không tuần tự), nhanh hơn RNN, dùng Dilated Convolutions để nắm bắt chuỗi độ dài hàng chục ngàn bước (vd: Âm thanh).